Course End Project- Enterprise Infrastructure Security

Securing Linux Servers using SSH Honeypot monitoring and IP blocking

By

Sharmila Vijayan

1. Introduction:

Most of the organizations frequently face unauthorized access attempts from against internal Linux systems, particularly via the Secure Shell (SSH) service. These attacks often originate from compromised internal hosts. This project focuses on designing and implementing a host-based SSH honeypot to detect, log, and analyze malicious access attempts within a segmented enterprise network.

2. Problem Statement:

Secure Defense, a cybersecurity services firm, is responsible for protecting internal Linux-based systems deployed within segmented enterprise networks. These systems are frequently targeted by unauthorized SSH access attempts originating from compromised hosts. Due to compliance and infrastructure limitations, the organization cannot rely on cloud-based monitoring tools or publicly exposed servers. The challenge is to:

- Detect unauthorized SSH access attempts
- Collect attacker behavior and metadata
- Reduce brute-force and block attacker IP address

3. Project Objective:

To design and implement a controlled security monitoring solution for Secure Defense against unauthorized SSH access attempts. The solution detects, analyzes, and mitigates SSH brute-force attacks on Linux systems by using honeypot-style monitoring, log analysis, SSH hardening techniques, and IP based access control mechanisms.

4. Scope of the Project

- Linux server
- SSH service configuration and monitoring
- Local logging and analysis
- Host-based intrusion prevention

5. System Architecture:

The honeypot system consists of:

- Create a SSH service to act as a honeypot.
- Prepare a virtual directory to view the logs.
- Expose the SSH port.
- Capturing brute force attack attempts.
- Logging Attackers IP address.
- Blocking malicious IP address.

All components operate locally without external dependencies.

6. Implementation Details:

6.1 SSH Honeypot Deployment

A controlled SSH service was configured to log all authentication attempts. Invalid login attempts were intentionally allowed to capture attacker behavior without granting shell access.

6.2 SSH Hardening Measures

Security configurations included:

- Disabling root login
- Limiting authentication retries
- Restricting allowed users

6.3 Logging and Monitoring

Authentication attempts were logged to centralized log files stored in a secured /reports directory with restricted permissions.

6.4 IP-Based Blocking

Repeated failed login attempts triggered automatic IP blocking to mitigate brute-force attacks while preserving honeypot data for analysis.

7. Testing and Results:

Testing involved simulated brute-force attacks from internal test machines. Results showed:

- Successful logging of all SSH attempts
- Accurate identification of repeated attack sources
- Effective blocking after defined thresholds
- No disruption to legitimate system services

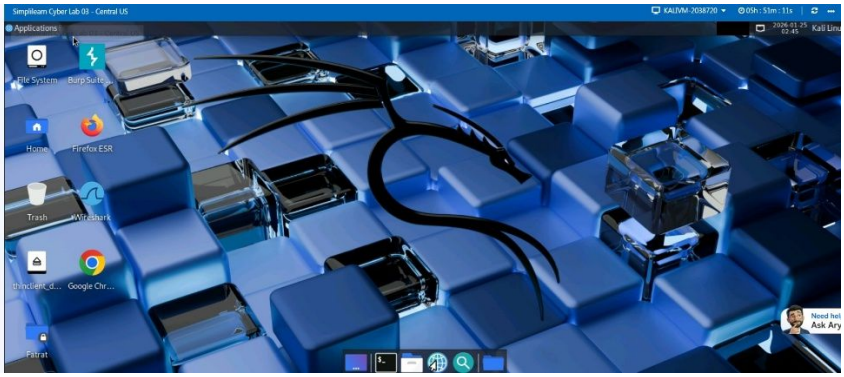
8. Steps to be followed to achieve the results:

1. Create a SSH service to act as a honeypot.
2. Prepare virtual directory to view the logs
3. Expose the SSH port.
4. Capture the brute force attack attempts.
5. Log Attackers IP address.
6. Block malicious IP address.

Step 1: Create a SSH service to act as a honeypot.

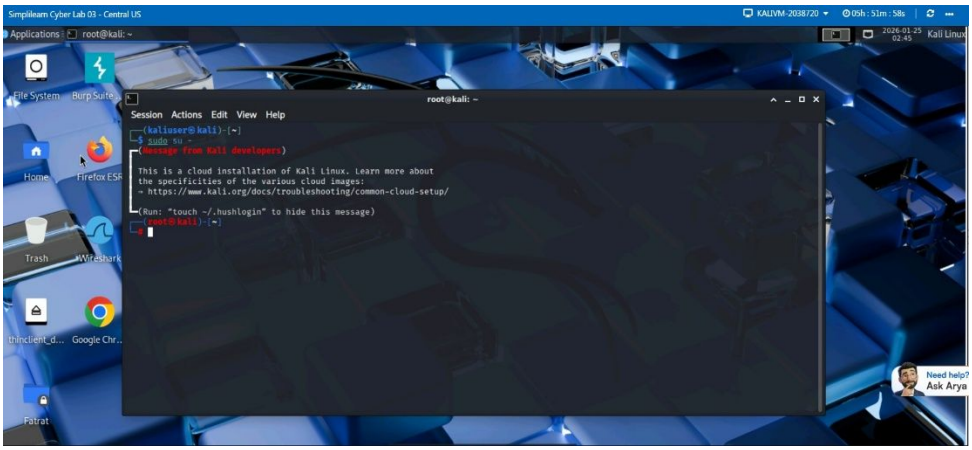
System Setup and Package Installation

1.1 Navigate to Kali Linux VM and open Terminal Emulator.



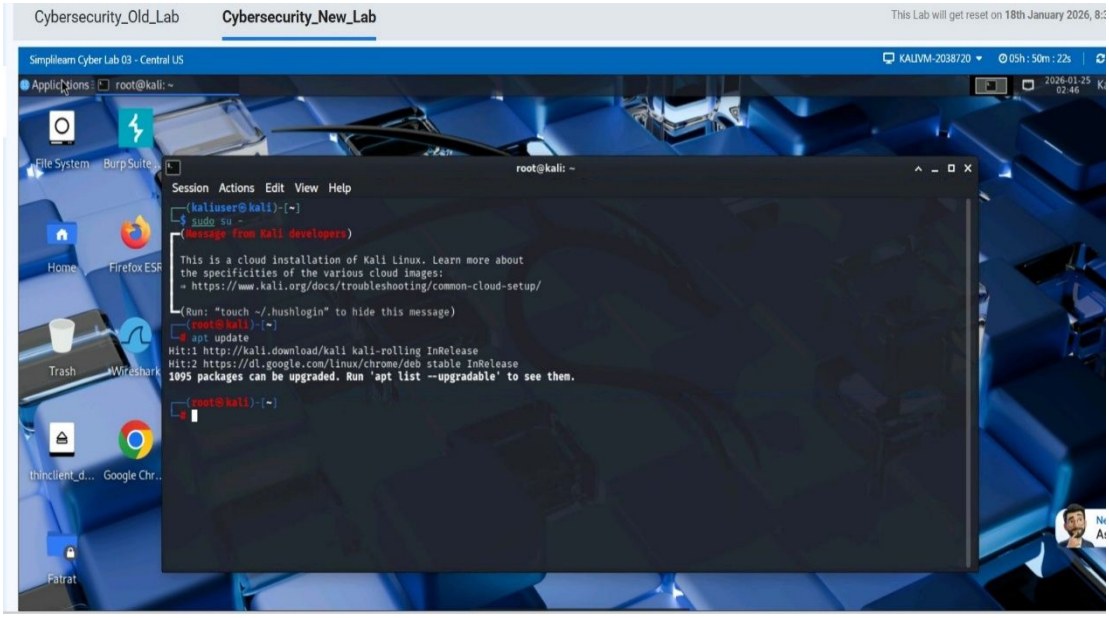
1.2 Run the following command to change to the root user.

sudo su



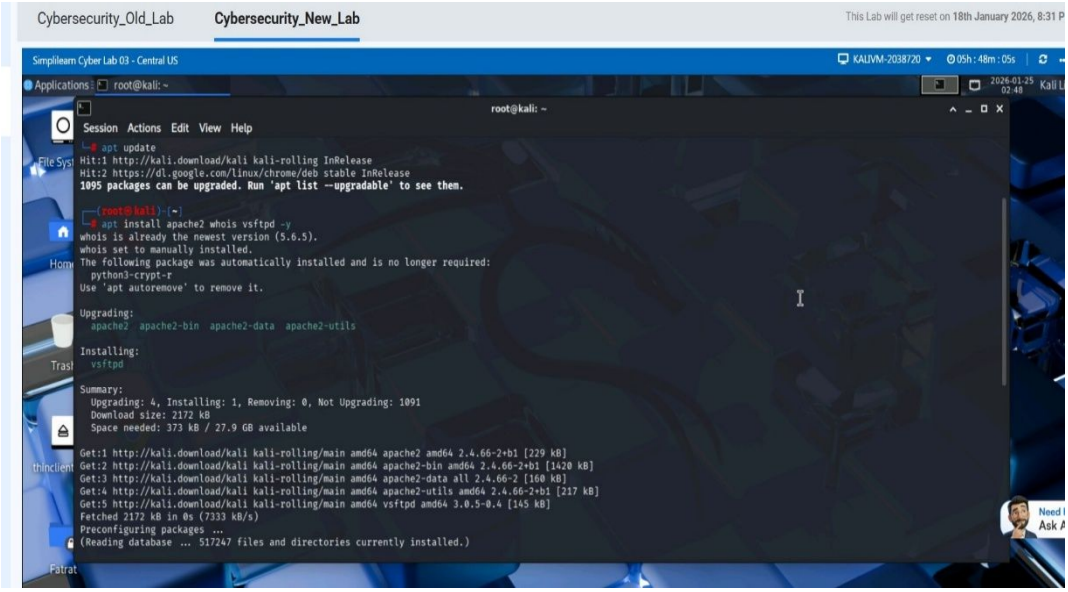
1.3 Run the following command to update package information:

apt install



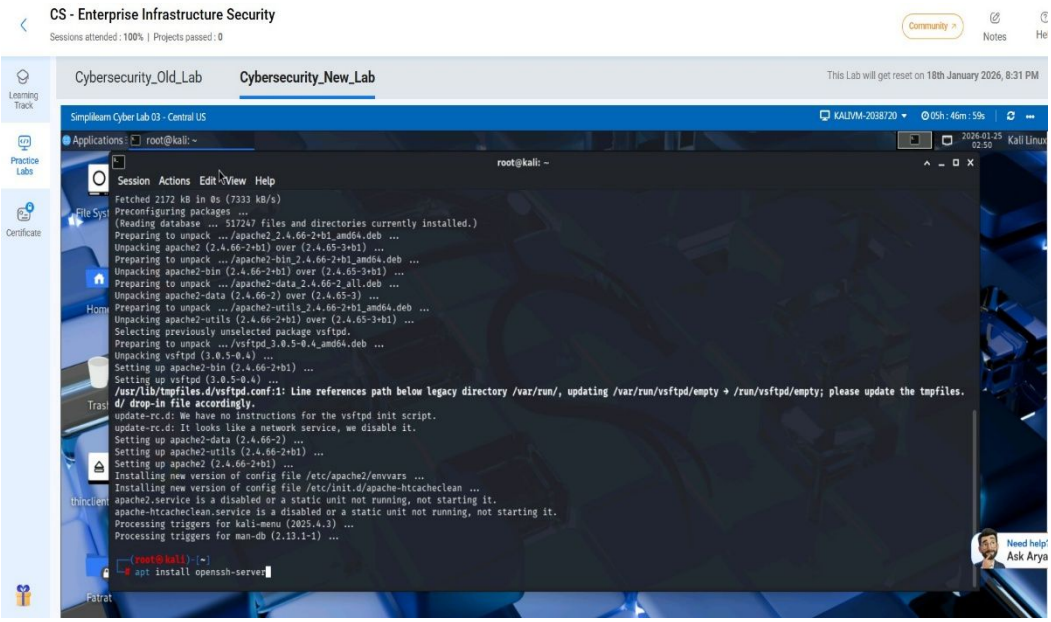
1.4. Run the following command to install Apache webserver, Network lookup tool and ftp server.

apt install apache2 whois vsftpd -y



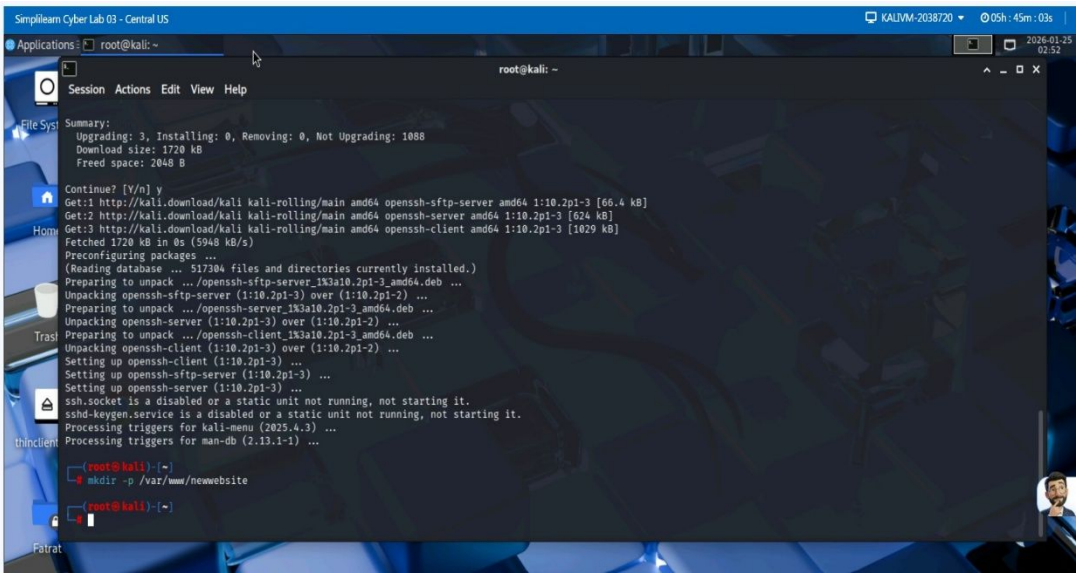
1.5 Run the following command to install the SSH server to allow remote secure login:

apt install openssh-server

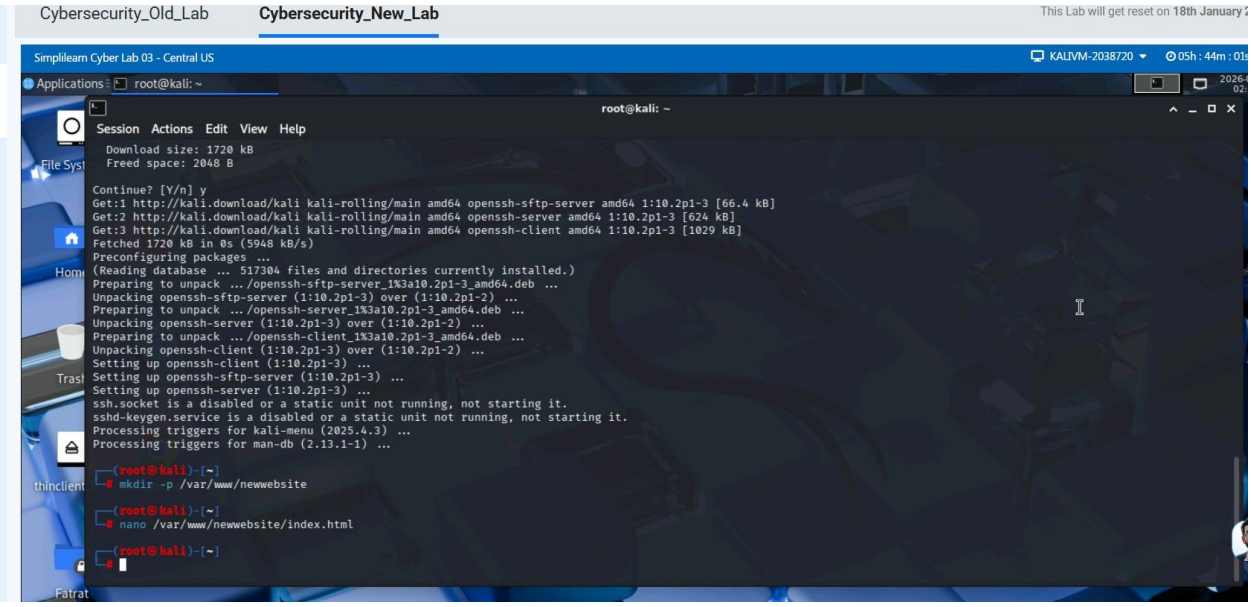


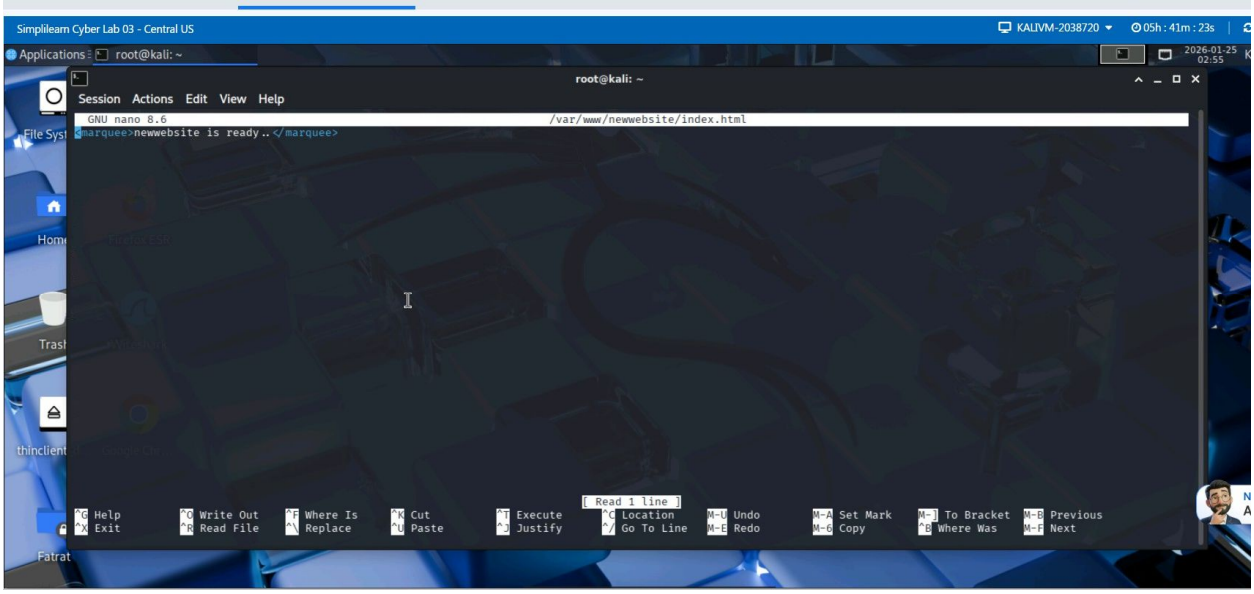
Web Server Configuration

1.6 Run the following command to create a directory for new website:
mkdir -p /var/www/newebsite

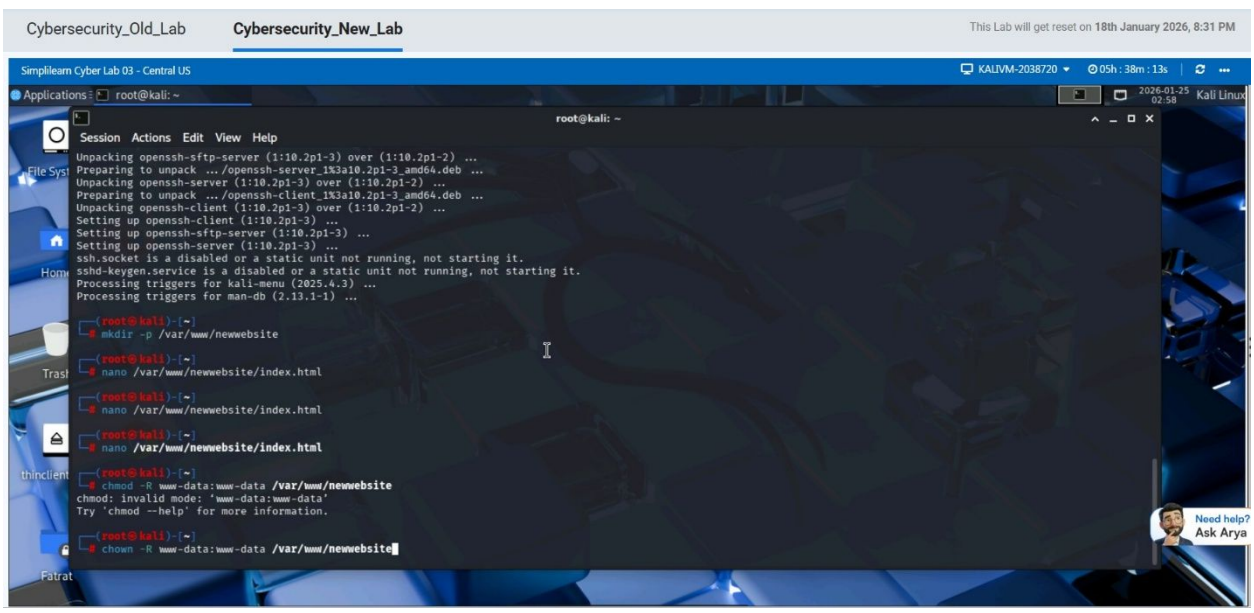


1.7 Run the following command to create an html file for website content.
nano /var/www/ newebsite /index.html

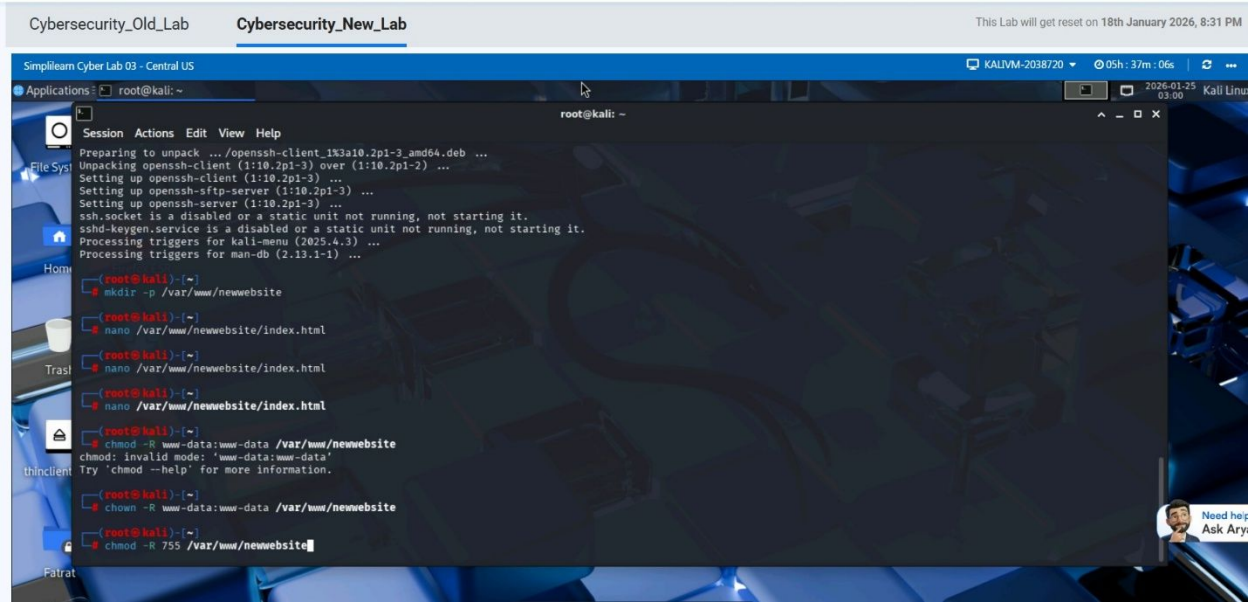




1.8 Run the following command to change permission to apache so it can read website files, and serves pages correctly:
chown -R www-data:www-data /var/www/newwebsite

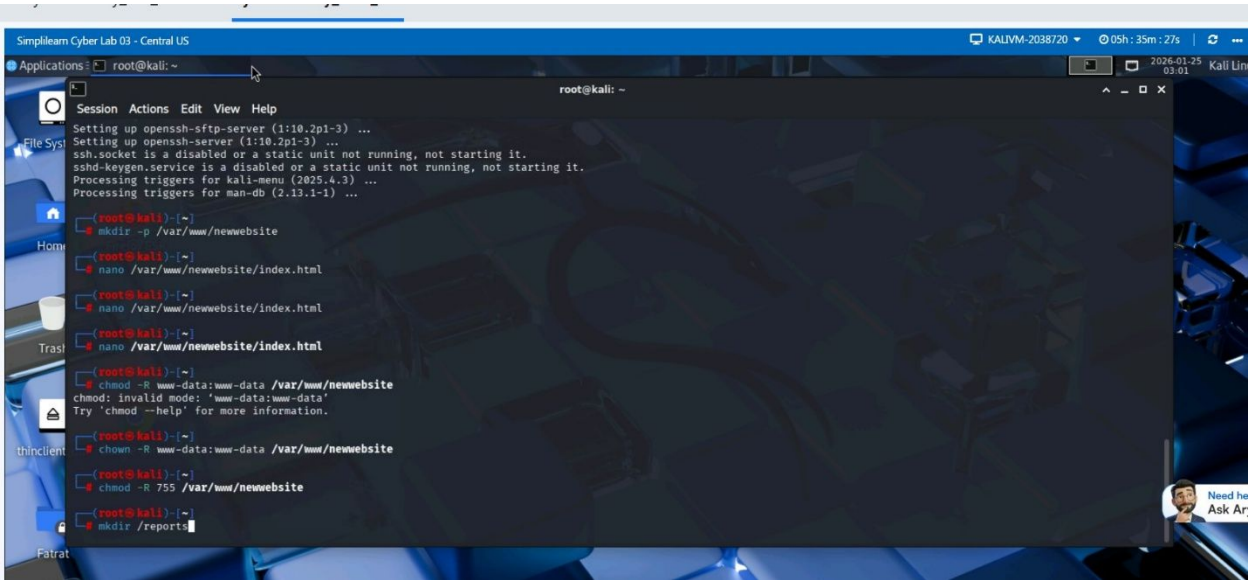


1.9 Run the following command to set permission so owners can edit, apache can read and access the directories, and does not allow others to edit it:
chmod -R 755 /var/www/ newwebsite

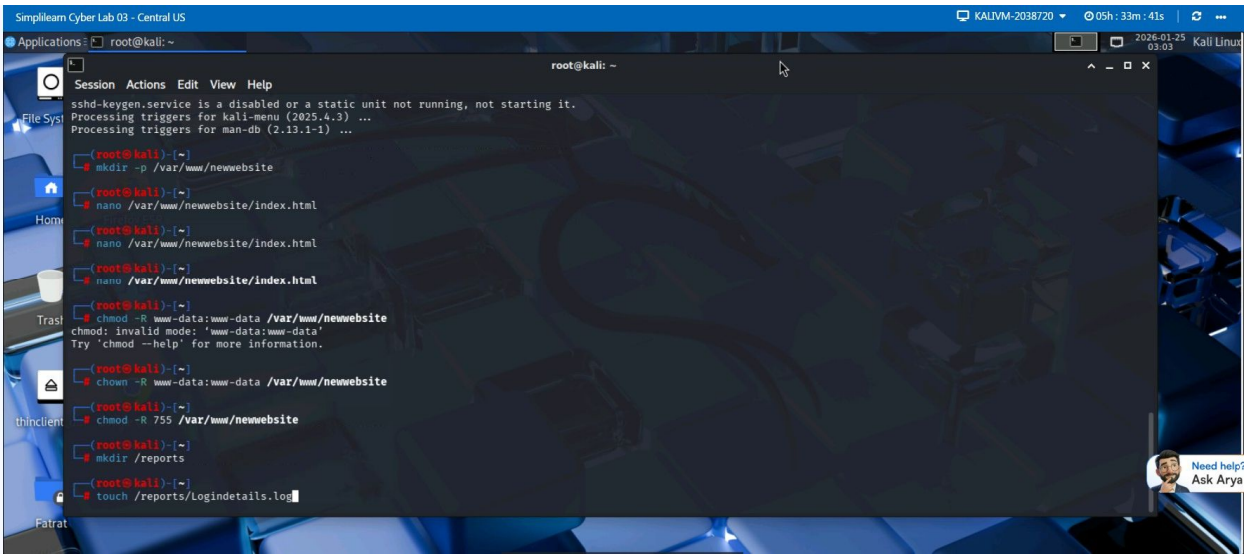


Step 2. Prepare virtual directory to view the logs

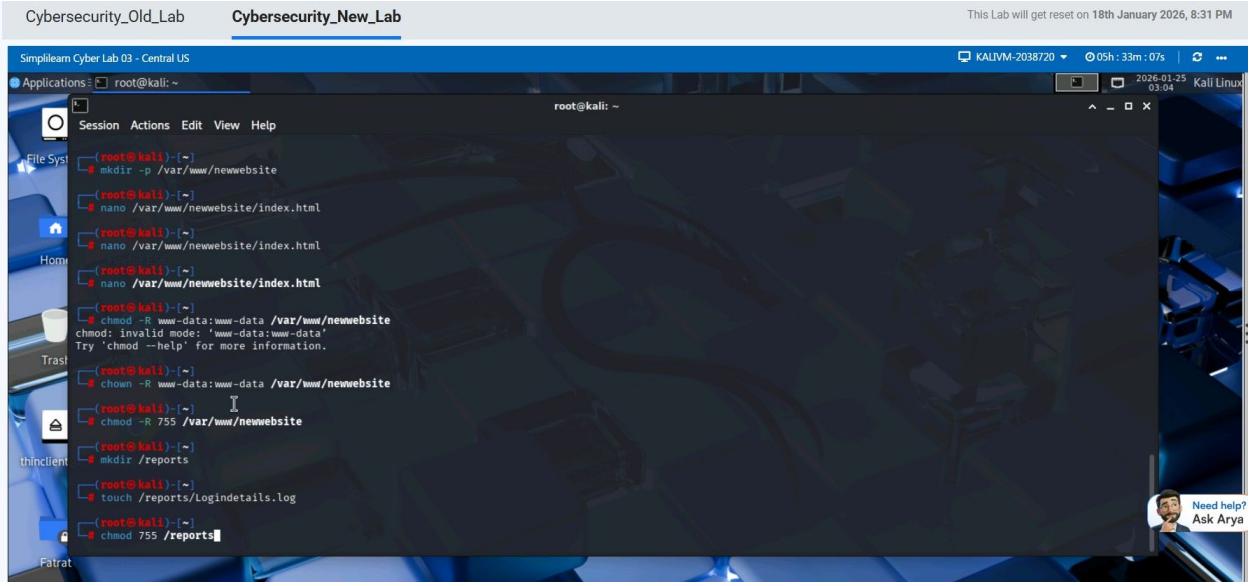
2.1 Run the following command to Creates a directory to store security or audit logs.
mkdir /reports



2.2 Run the following command to create an empty log file.
touch /reports/Logindetails.log

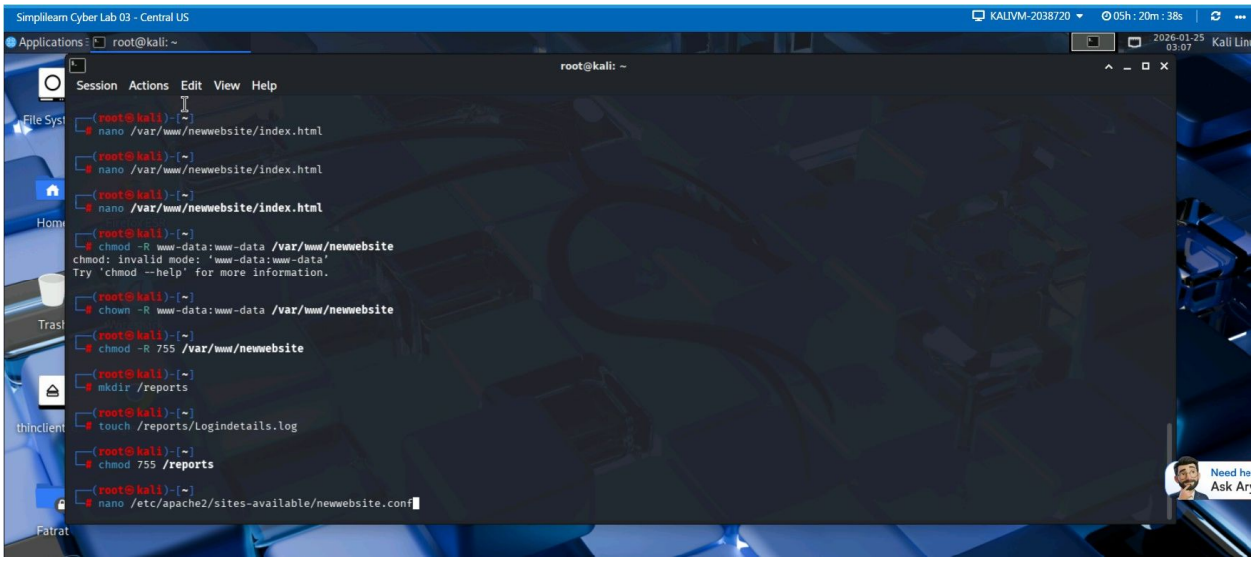


2.3 Run the following command to change the directory access permissions.
chmod 755 /reports



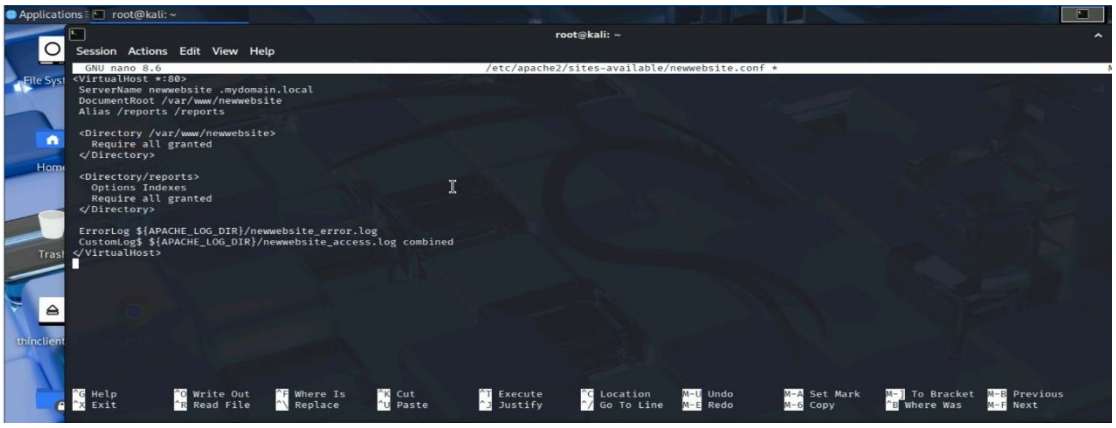
Step 3: Expose the SSH port

3.1 Run the following command to creates virtual host configuration file for the test website.
nano /etc/apache2/sites-available/newwebsite.conf

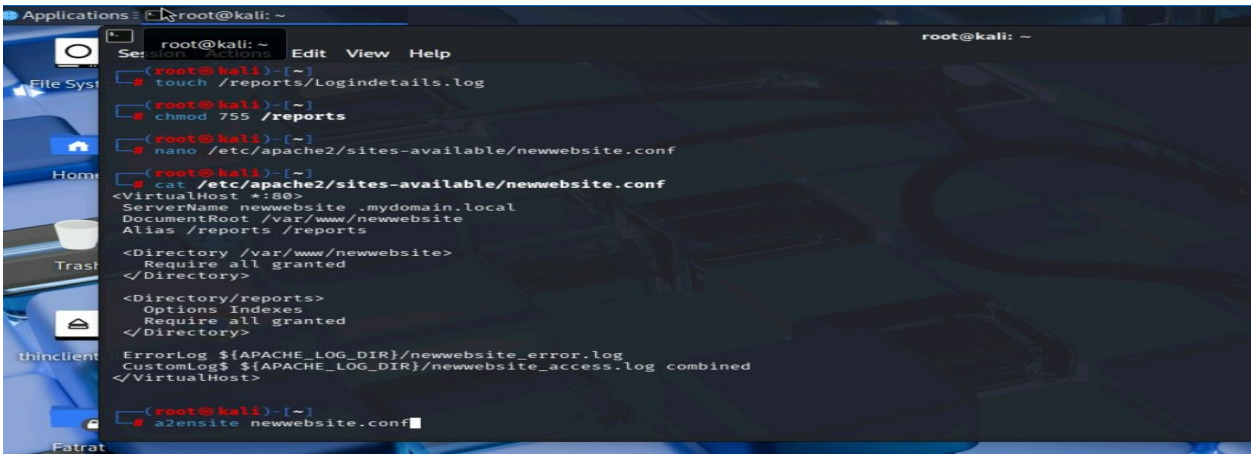


3.2 Add the following configuration on to the config file.

```
<VirtualHost *:80>
 ServerName newwebsite.mydomain.local
 DocumentRoot /var/www/ newwebsite
 Alias /reports /reports
 <Directory /var/www/ newwebsite >
 Require all granted
 </Directory>
 <Directory /reports>
 Options Indexes
 Require all granted
 </Directory>
 ErrorLog ${APACHE_LOG_DIR}/ newwebsite _error.log
 CustomLog ${APACHE_LOG_DIR}/ newwebsite _access.log combined
</VirtualHost>
```

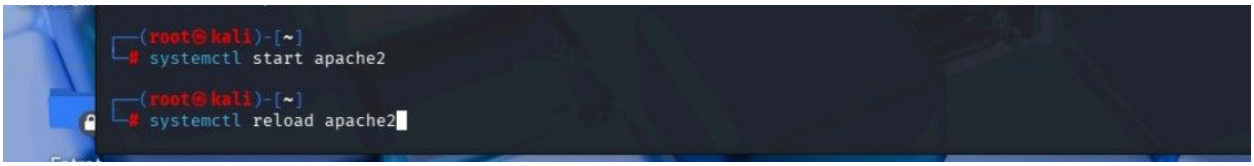


3.3 Run the following command to enable the new Apache site.
a**2ensite newwebsite.conf**

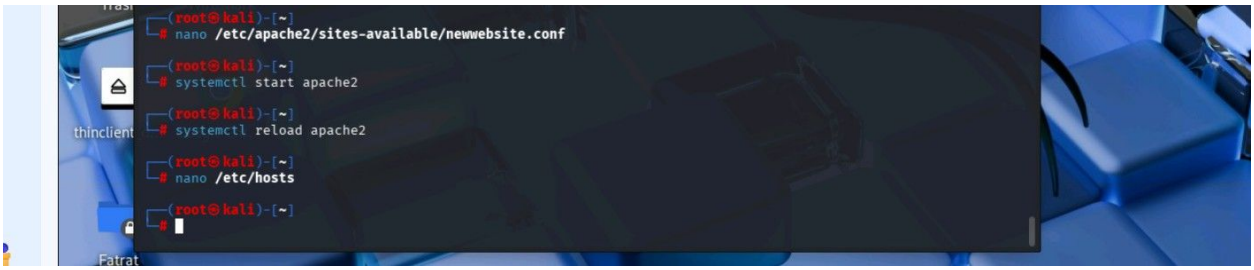


3.4 Run the following command to start the Apache service.
s**ystemctl start apache2**

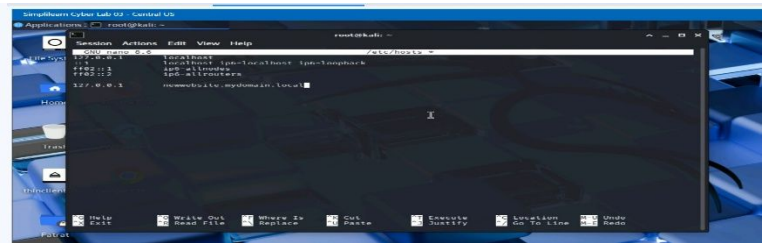
3.5 Run the following command to reload apache configuration without stopping the server.
systemctl reload apache2



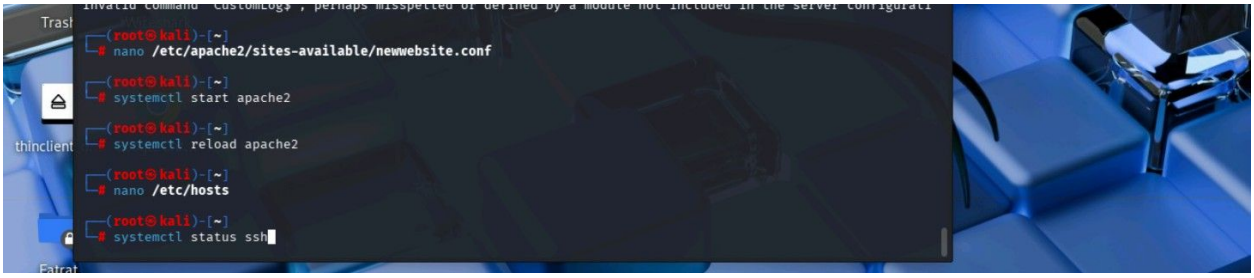
3.6 Run the following command to edit the hosts file.
nano /etc/hosts



3.7 Add the following to map local host and new website domain.
127.0.0.1 newwebsite.mydomain.local

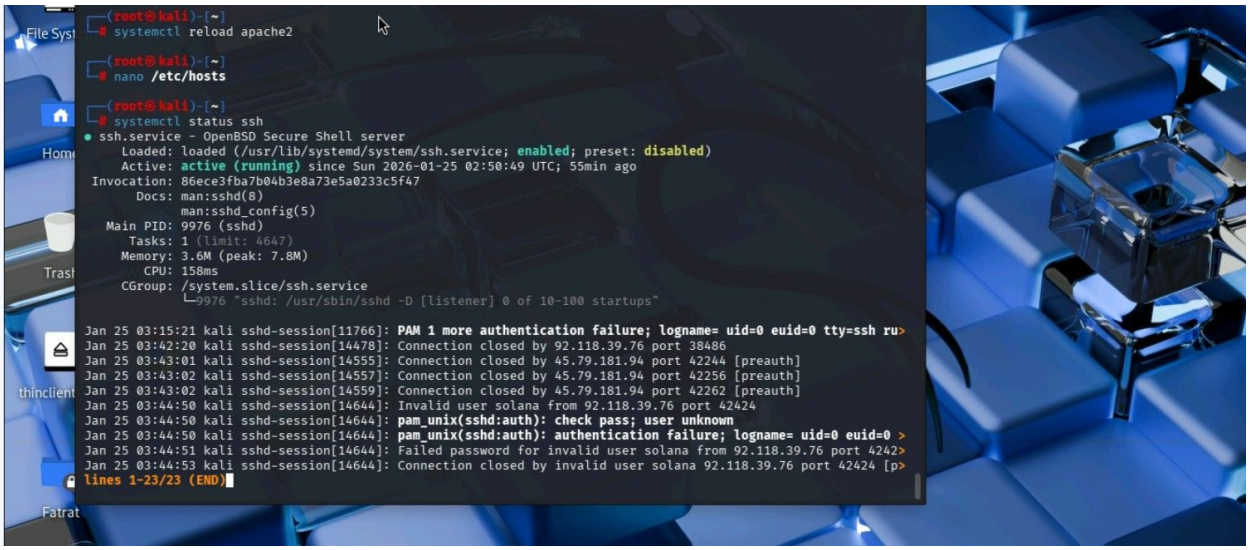


3.8 Run the following command checks whether the SSH service is running.
systemctl status ssh

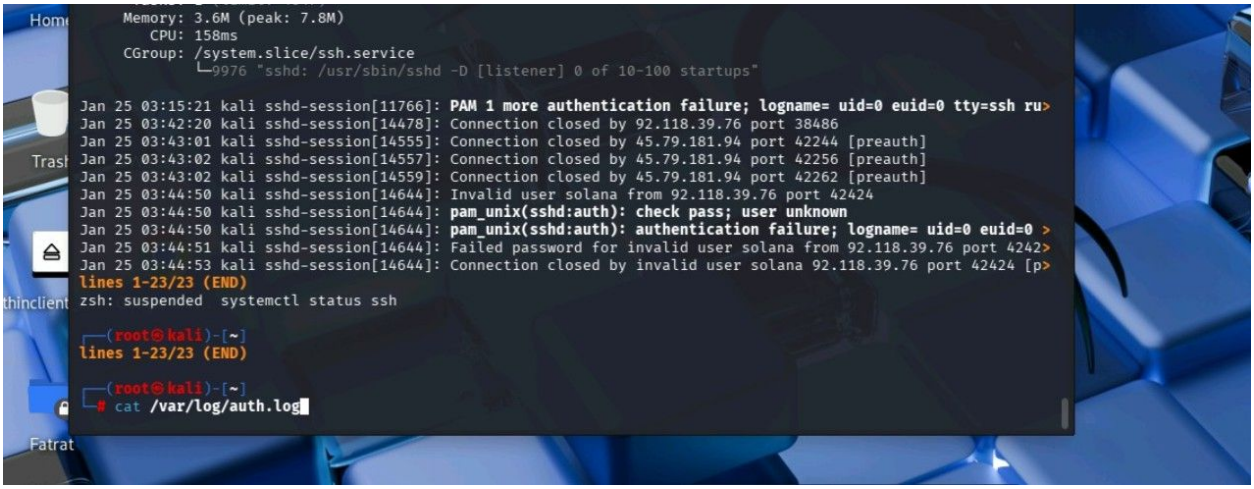


Step 4. Capture the brute force attack attempts.

4.1 Run the command to display authentication logs (SSH, sudo, login attempts).
cat /var/log/auth.log



4.2 Run the command to filters failed SSH login attempts.
grep "Failed password" /var/log/auth.log



```
2026-01-25T02:31:56.289474+00:00 kali sudo: pam_unix(sudo:session): session closed for user root
2026-01-25T02:31:56.298939+00:00 kali sudo:     root : PWD=/var/lib/waagent/run-command/download/0 ; USER=root
 ; ENV=DEBIAN_FRONTEND=noninteractive ; COMMAND=/usr/bin/apt-get -y install jq
2026-01-25T02:31:56.299079+00:00 kali sudo: pam_unix(sudo:session): session opened for user root(uid=0) by (ui
d=0)
2026-01-25T02:32:00.265833+00:00 kali sudo: pam_unix(sudo:session): session closed for user root
2026-01-25T02:32:00.276202+00:00 kali sudo:     root : PWD=/var/lib/waagent/run-command/download/0 ; USER=root
 ; ENV=DEBIAN_FRONTEND=noninteractive ; COMMAND=/usr/bin/apt-get -y install moreutils
2026-01-25T02:32:00.276347+00:00 kali sudo: pam_unix(sudo:session): session opened for user root(uid=0) by (ui
d=0)
2026-01-25T02:32:11.154583+00:00 kali sudo: pam_unix(sudo:session): session closed for user root
2026-01-25T02:32:11.170931+00:00 kali sudo:     root : PWD=/var/lib/waagent/run-command/download/0 ; USER=root
 ; COMMAND=/usr/bin/systemctl enable auditd
2026-01-25T02:32:11.171068+00:00 kali sudo: pam_unix(sudo:session): session opened for user root(uid=0) by (ui
d=0)
2026-01-25T02:32:12.066191+00:00 kali sudo: pam_unix(sudo:session): session closed for user root
2026-01-25T02:32:12.085080+00:00 kali sudo:     root : PWD=/var/lib/waagent/run-command/download/0 ; USER=root
 ; COMMAND=/usr/bin/systemctl start auditd
2026-01-25T02:32:12.085182+00:00 kali sudo: pam_unix(sudo:session): session opened for user root(uid=0) by (ui
d=0)
2026-01-25T02:32:12.108649+00:00 kali sudo: pam_unix(sudo:session): session closed for user root
2026-01-25T02:32:12.123916+00:00 kali sudo:     root : PWD=/var/lib/waagent/run-command/download/0 ; USER=root
 ; COMMAND=/usr/bin/tee -a /etc/audit/rules.d/audit.rules
2026-01-25T02:32:12.124043+00:00 kali sudo: pam_unix(sudo:session): session opened for user root(uid=0) by (ui
d=0)
2026-01-25T02:32:12.125339+00:00 kali sudo: pam_unix(sudo:session): session closed for user root
2026-01-25T02:32:12.144157+00:00 kali sudo:     root : PWD=/var/lib/waagent/run-command/download/0 ; USER=root
 ; COMMAND=/usr/bin/tee -a /etc/audit/rules.d/audit.rules
2026-01-25T02:32:12.144215+00:00 kali sudo: pam_unix(sudo:session): session opened for user root(uid=0) by (ui
d=0)
2026-01-25T02:32:12.145232+00:00 kali sudo: pam_unix(sudo:session): session closed for user root
2026-01-25T02:32:12.162729+00:00 kali sudo:     root : PWD=/var/lib/waagent/run-command/download/0 ; USER=root
```

```
2026-01-25T03:44:53.709152+00:00 kali sshd-session[14644]: Connection closed by invalid user solana 92.118.39.
76 port 42424 [preauth]
2026-01-25T03:45:01.134872+00:00 kali CRON[14652]: pam_unix(cron:session): session opened for user root(uid=0)
 by root(uid=0)
2026-01-25T03:45:01.135273+00:00 kali CRON[14651]: pam_unix(cron:session): session opened for user root(uid=0)
 by root(uid=0)
2026-01-25T03:45:01.137889+00:00 kali CRON[14652]: pam_unix(cron:session): session closed for user root
2026-01-25T03:45:01.137992+00:00 kali CRON[14651]: pam_unix(cron:session): session closed for user root
2026-01-25T03:47:00.312380+00:00 kali sshd-session[14772]: Invalid user sol from 92.118.39.76 port 38542
2026-01-25T03:47:00.479537+00:00 kali sshd-session[14772]: pam_unix(sshd:auth): check pass; user unknown
2026-01-25T03:47:00.479638+00:00 kali sshd-session[14772]: pam_unix(sshd:auth): authentication failure; lognam
e= uid=0 euid=0 tty=ssh ruser= rhost=92.118.39.76
2026-01-25T03:47:02.424675+00:00 kali sshd-session[14772]: Failed password for invalid user sol from 92.118.39
.76 port 38542 ssh2
2026-01-25T03:47:02.575521+00:00 kali sshd-session[14772]: Connection closed by invalid user sol 92.118.39.76
port 38542 [preauth]

┌──(root㉿kali)-[~]
└─# grep "Failed password" /var/log/auth.log
```

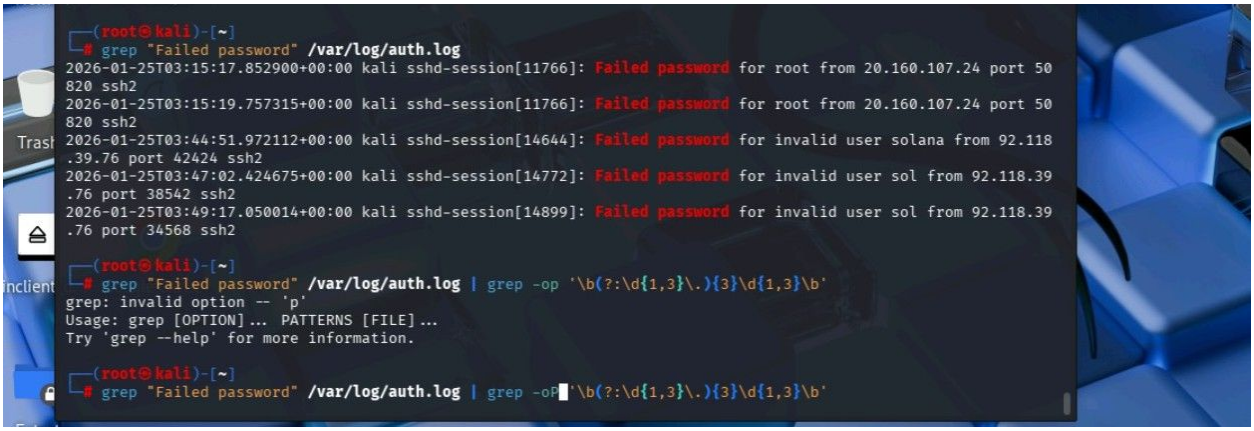
```
2026-01-25T03:47:02.575521+00:00 kali sshd-session[14772]: Connection closed by invalid user sol 92.118.39.76
port 38542 [preauth]

┌──(root㉿kali)-[~]
└─# grep "Failed password" /var/log/auth.log
2026-01-25T03:15:17.852900+00:00 kali sshd-session[11766]: Failed password for root from 20.160.107.24 port 50
820 ssh2
2026-01-25T03:15:19.757315+00:00 kali sshd-session[11766]: Failed password for root from 20.160.107.24 port 50
820 ssh2
2026-01-25T03:44:51.972112+00:00 kali sshd-session[14644]: Failed password for invalid user solana from 92.118
.39.76 port 42424 ssh2
2026-01-25T03:47:02.424675+00:00 kali sshd-session[14772]: Failed password for invalid user sol from 92.118.39
.76 port 38542 ssh2
2026-01-25T03:49:17.050014+00:00 kali sshd-session[14899]: Failed password for invalid user sol from 92.118.39
.76 port 34568 ssh2

┌──(root㉿kali)-[~]
└─#
```

Step 5. Log Attackers IP address.

5.1 Run the command to extract attacker IP addresses.
grep "Failed password" /var/log/auth.log | grep -oP '\b(?:\ d{1,3}\.){3}\d{1,3}\b'



5.2 Run the command to sort and remove duplicate IPs to identify unique attackers.
grep "Failed password" /var/log/auth.log | grep -oP '\b(?:\ d{1,3}\.){3}\d{1,3}\b' | sort -u

Step 6. Block all malicious IP address.

6.1 Run the command create a script to track SSH attacks.
nano /root/b_track_ssh_daily

6.2 Add the SSH tracking script.

```
#!/bin/bash
# -----------------------------------------------------
# Script Name : SSH Failed Login Tracker
# Purpose : Identify IP addresses involved in
# failed SSH login attempts and generate
# a readable report for analysis.
# Log Source : /var/log/auth.log (Kali Linux)
# Output : /reports/ssh_failed_logins_report.txt
# -----------------------------------------------------
# Step 1:
# Search the SSH authentication log for failed password attempts.
# Extract only IPv4 addresses using a regular expression.
# Sort and keep only unique IP addresses.
grep "Failed password" /var/log/auth.log \
| grep -E -o "([0-9]{1,3}\.){3}[0-9]{1,3}" \
| sort -u > /tmp/ssh_attackers.txt
# Step 2:
# Clear the previous report file so each execution
# starts with a fresh report.
> /reports/ssh_failed_logins_report.txt
# Step 3:
# Read each attacker IP address from the temporary file.
# For each IP:
# - Perform a WHOIS lookup
# - Extract the country information
# - Append the IP and country details to the report
while read ip; do
 country=$(whois $ip | grep -i country | head -n1)
 echo "$ip $country" >> /reports/ssh_failed_logins_report.txt
done < /tmp/ssh_attackers.txt
# End of Script
# The final report can be accessed via:
# http:// newwebsite.mydomain.local/reports/ssh_failed_logins_report.txt
```

```
Session  Actions  Edit  View  Help
  GNU nano 8.6                          /root/b_track_ssh_daily *
#!/bin/bash
# ------------------------------------------------------------
# Script Name : SSH Failed Login Tracker
# Purpose : Identify IP addresses involved in
# failed SSH login attempts and generate
# a readable report for analysis.
# Log Source : /var/log/auth.log (Kali Linux)
# Output : /reports/ssh_failed_logins_report.txt
# ------------------------------------------------------------
# Step 1:
# Search the SSH authentication log for failed password attempts.
# Extract only IPv4 addresses using a regular expression.
# Sort and keep only unique IP addresses.
grep "Failed password" /var/log/auth.log \
| grep -E -o "([0-9]{1,3}\.){3}[0-9]{1,3}" \
| sort -u > /tmp/ssh_attackers.txt
# Step 2:
# Clear the previous report file so each execution
# starts with a fresh report.
> /reports/ssh_failed_logins_report.txt
# Step 3:
# Read each attacker IP address from the temporary file.
# For each IP:
# - Perform a WHOIS lookup
# - Extract the country information
# - Append the IP and country details to the report
while read ip; do
 country=$(whois $ip | grep -i country | head -n1)

^G Help      ^O Write Out  ^F Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo
```

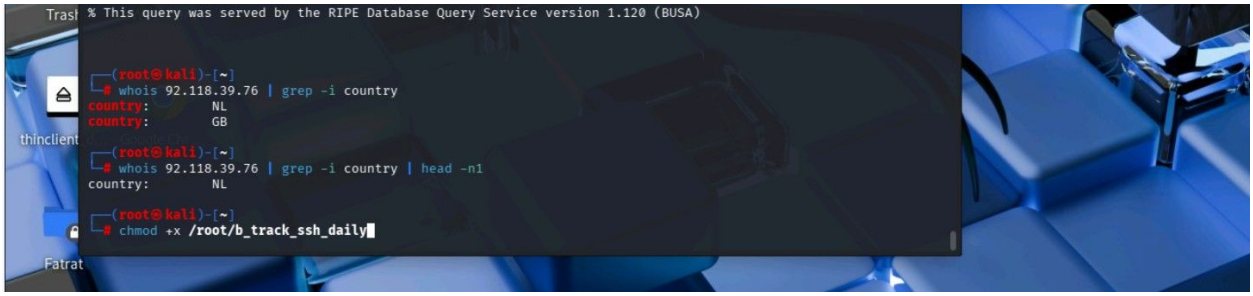
```
root@kali: ~
Session  Actions  Edit  View  Help
  GNU nano 8.6                          /root/b_track_ssh_daily *
# Log Source : /var/log/auth.log (Kali Linux)
# Output : /reports/ssh_failed_logins_report.txt
# ------------------------------------------------------------
# Step 1:
# Search the SSH authentication log for failed password attempts.
# Extract only IPv4 addresses using a regular expression.
# Sort and keep only unique IP addresses.
grep "Failed password" /var/log/auth.log \
| grep -E -o "([0-9]{1,3}\.){3}[0-9]{1,3}" \
| sort -u > /tmp/ssh_attackers.txt
# Step 2:
# Clear the previous report file so each execution
# starts with a fresh report.
> /reports/ssh_failed_logins_report.txt
# Step 3:
# Read each attacker IP address from the temporary file.
# For each IP:
# - Perform a WHOIS lookup
# - Extract the country information
# - Append the IP and country details to the report
while read ip; do
 country=$(whois $ip | grep -i country | head -n1)
 echo "$ip $country" >> /reports/ssh_failed_logins_report.txt
done < /tmp/ssh_attackers.txt
# End of Script
# The final report can be accessed via:
# http://testsite.mydomain.local/reports/ssh_failed_logins_report.txt

^G Help      ^O Write Out  ^F Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo
```

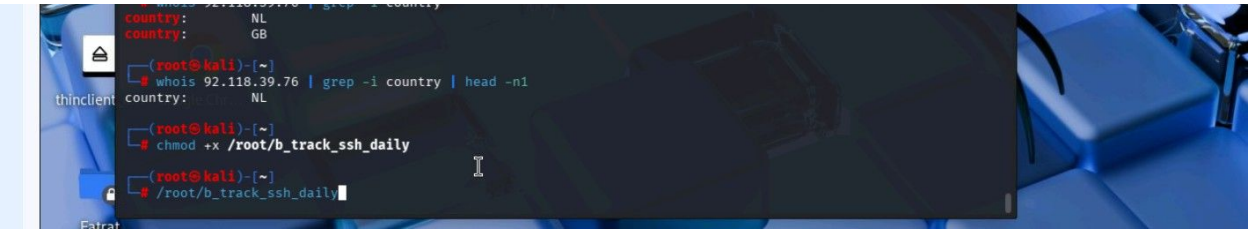
6.3 Run the command retrieve registration details of an attacking IP.
whois 20.160.107.24

6.4 Run the command to extract country information to show the first matching country entry.
whois 20.160.107.24| grep -i country | head -n1

6.5 Run the command to Make the SSH script executable.
chmod +x /root/b_track_ssh_daily



6.6 Use the command to run the SSH tracking script.
/root/b_track_ssh_daily

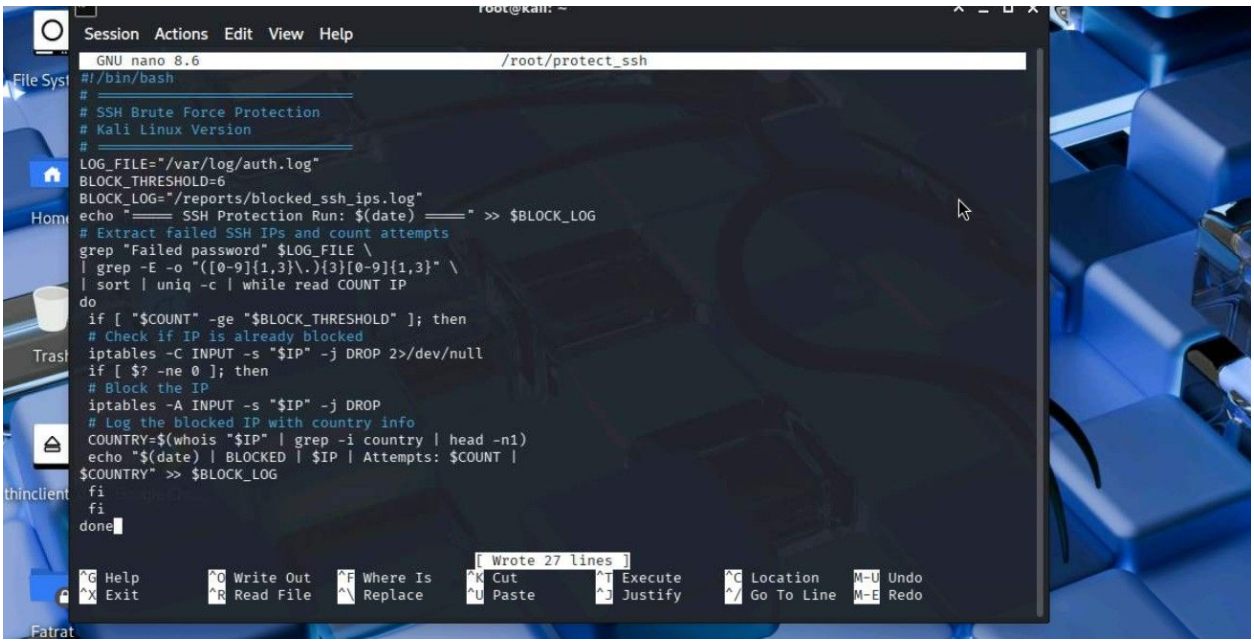


6.7 Run the command to open **protect_ssh**.
nano /root/protect_ssh

6.8 Enter the following script to defend SSH (typically via firewall rules).

```
#!/bin/bash
# ==============================
# SSH Brute Force Protection
# Kali Linux Version
# ==============================
LOG_FILE="/var/log/auth.log"
BLOCK_THRESHOLD=6
BLOCK_LOG="/reports/blocked_ssh_ips.log"
echo "===== SSH Protection Run: $(date) =====" >> $BLOCK_LOG
# Extract failed SSH IPs and count attempts
grep "Failed password" $LOG_FILE \
| grep -E -o "([0-9]{1,3}\.){3}[0-9]{1,3}" \
| sort | uniq -c | while read COUNT IP
do
 if [ "$COUNT" -ge "$BLOCK_THRESHOLD" ]; then
 # Check if IP is already blocked
 iptables -C INPUT -s "$IP" -j DROP 2>/dev/null
 if [ $? -ne 0 ]; then
 # Block the IP
```

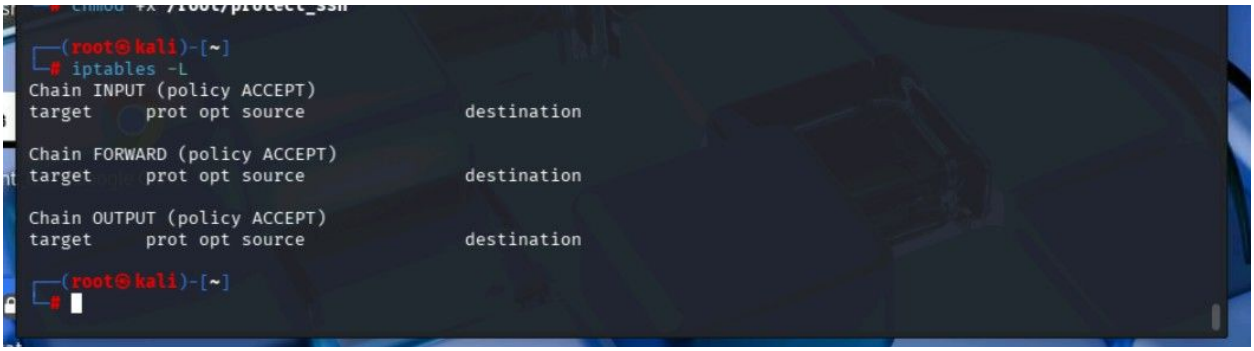
```
 iptables -A INPUT -s "$IP" -j DROP
 # Log the blocked IP with country info
 COUNTRY=$(whois "$IP" | grep -i country | head -n1)
 echo "$(date) | BLOCKED | $IP | Attempts: $COUNT |
$COUNTRY" >> $BLOCK_LOG
 fi
 fi
done
```



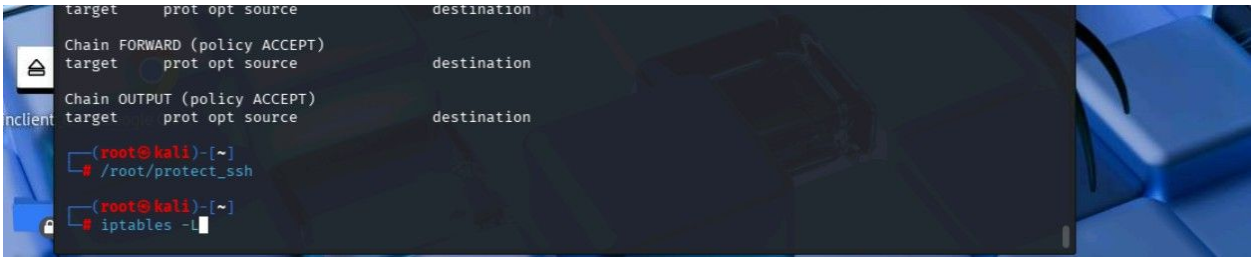
6.9 Run the following command to Allows execution.

chmod +x /root/protect_ssh

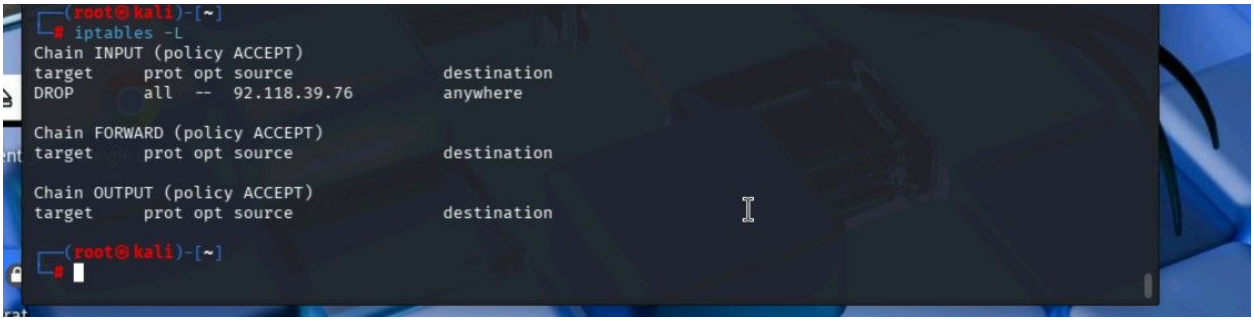
6.10 Run the following command to Lists current firewall rules.
iptables -L



6.11 Run the following command to execute protect_ssh to apply SSH protection rules
/root/protect_ssh



6.12 Run the Verifies that firewall rules were successfully applied.
iptables -L



By following these steps, we can successfully host a SSH service to act as a honeypot. Using logs, all brute force attack attempts can be captured. From the logs the attacker ips can be filtered and blocked, thereby hardening the SSH service.

References:

1. Simplilearn Project Mentoring Class
2. Security +Guide to Network Fundamentals

Thank You